SISTEMA DE PONTO DE VENDA (PDV) WEB

PDV Web

Documentação do Projeto de Software

Solução de ponto de venda em nuvem: controle de vendas, estoque, caixa, descontos e comissões, com segurança por perfil e qualidade comprovada por testes.

Integrantes

Hugo de Oliveira Guimarães

Jean Wictor

Cesar Franco

Backend `pdv-web-api` 2.3.0 · Documentação versão 2.0 · 02/11/2026

Clique em qualquer item do sumário para ir direto à seção.

Sumário

Apresentação do Projeto **3**

1. Levantamento de Requisitos **6**

1.1 Requisitos Funcionais 7

1.2 Requisitos Não Funcionais 8

1.3 Diagrama de Caso de Uso 9

1.4 Detalhamento de Caso de Uso 10

1.4.1 Dicionário de Casos de Uso 10

1.4.2 Especificação e Descrição de Casos de Uso 12

1.5 Perfis de Acesso 14

1.6 Regras de Negócio e Validações 14

2. Projeto **16**

2.1 Diagrama Entidade-Relacionamento 17

2.1.1 Modelo Lógico do Banco de Dados 18

2.2 Dicionário de Dados 20

2.3 Tecnologias 25

2.4 Contrato da API 26

2.5 Segurança 27

3. Qualidade e Testes **28**

3.1 Processo e Plano de Testes 29

3.2 Resultados da Execução 29

3.3 Defeitos 31

3.4 Cenários BDD e Rastreabilidade 31

Apresentação do Projeto

O **PDV Web** é uma solução de ponto de venda para o varejo, focada em processar vendas com rapidez, controlar estoque e caixa com rigor e impedir perdas financeiras por descontos indevidos, vendas sem saldo ou cancelamentos sem controle. O sistema opera como aplicação web, containerizada em Docker e preparada para hospedagem em nuvem (AWS EC2), com duas visões de uso: **Vendedor** e **Gerente**.

O problema

Divergência entre estoque físico e sistema, descontos concedidos sem autorização, comissões calculadas à mão e nenhum registro de quem fez cada operação sensível.

O objetivo

Uma frente de caixa web com controle de acesso por perfil, estoque atualizado em tempo real, caixa com abertura, sangria e fechamento, e comissão automática por vendedor.

A solução

Uma API REST segura (Spring Boot + PostgreSQL), executando em containers Docker, com todas as regras de negócio no servidor e 320 execuções de testes automatizados aprovadas.

Propósito, justificativa e status

Propósito	Otimizar o processo de vendas no varejo, com controle rigoroso de estoque e fluxo de caixa e atualização instantânea em nuvem.
Objetivo geral	Criar uma solução web de PDV eficiente para operação rápida em loja física e gestão financeira e operacional centralizada.
Justificativa	O comércio precisa de uma frente de caixa web flexível que elimine divergências entre o estoque físico e o sistema, automatize os acertos de comissões e restrinja a concessão indiscriminada de descontos, garantindo a integridade financeira do negócio.
Status	Em andamento — início em 20/06/2026, término previsto em 30/06/2027. Backend concluído e testado (Ciclos 1, 1.1 e 1.2); próxima etapa: front-end e implantação na AWS.
Equipe	Hugo de Oliveira Guimarães, Jean Wictor e Cesar Franco
Partes interessadas	Gerentes de loja, vendedores/operadores de caixa, suporte técnico e administradores
Custo de referência	R$ 5.000,00 (infraestrutura de nuvem AWS EC2 e ambiente de containers)

Objetivos específicos atendidos

Objetivo (Plano de Projeto, 1.3)	Como foi atendido	Situação
Controle de acesso seguro com CPF e senha	Login por CPF, senha em hash BCrypt, token JWT de 24 h	**Atendido**
Saldo de estoque e alertas de nível crítico em tempo real	Baixa na própria transação da venda; alerta quando estoque ≤ mínimo	**Atendido***
Impedir vendas com saldo insuficiente ou zerado	Validação com bloqueio de linha no banco, inclusive para vendas simultâneas	**Atendido**
Cancelamentos, estornos e descontos diretos só com o Gerente	Perfis verificados no filtro de segurança e no serviço; senha do gerente para desconto do vendedor	**Atendido**
Cálculo automático da comissão por vendedor	Percentual do vendedor congelado em cada venda; relatório por período	**Atendido**

* O backend entrega o alerta (na resposta da venda e na consulta de estoque). A notificação visual nas telas depende do front-end.

Escopo

INCLUÍDO

- Login, sessão e troca de senha
- Abertura e fechamento de caixa, sangria e suprimento
- Frente de caixa: venda por código/produto, cliente PF/PJ ou consumidor padrão, 5 formas de pagamento
- Cupons de desconto e desconto direto do gerente
- Cancelamento com estorno automático
- Catálogo, clientes, funcionários e comissões
- LOG de auditoria de todas as ações sensíveis
- Containerização com Docker Compose (alvo AWS EC2)

FORA DESTA FASE

- Integração TEF / gateway de cartão (pagamentos são simulados)
- Emissão de NF-e / NFC-e
- Interface web (front-end) — próxima etapa
- Reserva de estoque no carrinho (estoque tripartido)

Premissas, restrições e riscos

Premissas

- Autenticação por CPF e senha obrigatória desde o primeiro acesso
- Conexão contínua com a internet/nuvem para atualização simultânea dos dados
- Dois atores principais: Vendedor e Gerente

Restrições

- Cancelamento de vendas concluídas é exclusivo do Gerente
- Não são permitidas vendas com estoque zerado ou acima do disponível

Riscos e mitigação

- **Segurança:** acesso indevido → CPF/senha com hash, JWT, perfis
- **Estoque:** vendas simultâneas → bloqueio de linha no banco
- **Financeiro:** descontos indevidos → só por cupom ou autorização do Gerente

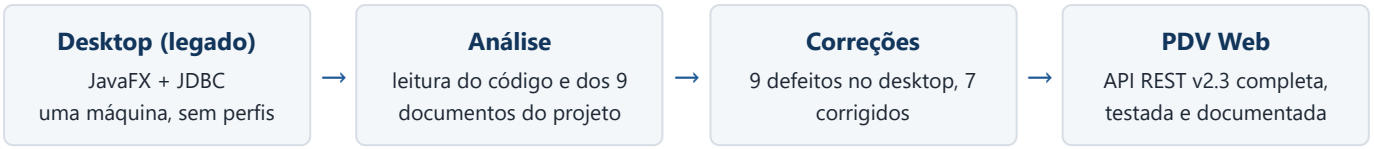
O projeto em números

16 casos de uso implementados (UC01–UC16)	**40** rotas na API REST	**320/320** execuções de testes automatizados aprovadas	**96,6%** das linhas de código cobertas por testes
8 validações de sistema (VAL-01 a VAL-08)	**11** tabelas com restrições e gatilhos de integridade	**100%** dos erros simulados detectados (teste de mutação)	**190 ms** p95 com 20 operadores simultâneos; limite: 3 s

Do sistema desktop ao PDV Web

O projeto começou como um **sistema desktop em JavaFX** ligado diretamente ao PostgreSQL. Ele já tinha cadastro de clientes (com busca automática de CNPJ), produtos, vendedores, tela de vendas com carrinho, histórico e relatório de

comissões. A documentação do projeto, porém, pede uma **aplicação web** com API REST, perfis de acesso, controle de caixa e auditoria.



Defeitos encontrados e corrigidos no desktop

ID	Problema	Gravidade	Correção
DEF-DSK-001	O estoque podia ficar negativo, inclusive com duas vendas ao mesmo tempo	Crítica	Baixa condicional no banco + validação no carrinho
DEF-DSK-002	Telas de clientes, produtos e vendedores não abriam fora do Windows / em JAR	Alta	Nomes dos arquivos de tela corrigidos
DEF-DSK-003	Relatório por vendedor sempre vazio (coluna inexistente)	Média	Consulta SQL corrigida
DEF-DSK-004	Editar cliente perdia bairro, estado, UF e CEP; a tela não tinha a opção de editar	Média	UPDATE completo + botão Atualizar
DEF-DSK-006	Não existia script do banco no repositório	Média	`database/schema.sql`
DEF-DSK-005/008	Comando de execução documentado não funcionava; recurso de banco não fechado	Baixa	Plugin JavaFX, Maven Wrapper e try-with-resources
DEF-DSK-007/009	Valores em `double`; CPF `000.000.000-00` aceito em qualquer cadastro	Média Baixa	Mantidos no legado; resolvidos no PDV Web

> O desktop permanece como **versão legada**. Os valores monetários em `double` e a exceção do CPF `000.000.000-00` foram mantidos nele e resolvidos no PDV Web, que usa `BigDecimal` e reserva esse documento ao "Consumidor Padrão". O reteste manual das correções (RT-DSK-01 a 07) está roteirizado e pendente.

Documentação analisada

Plano de Projeto, Regras de Sistema, Tabela de Planejamento, Documentação do Projeto, Casos de Uso, Dicionário de Dados, fluxo do usuário (DER), Documentação da API v2.3 e o Tutorial de Cenários de Testes. Onde os documentos divergiam, a decisão foi registrada no ADR-001 (`docs/arquitetura/`).

1

Levantamento de Requisitos

1.1 Requisitos Funcionais

1.2 Requisitos Não Funcionais

1.3 Diagrama de Caso de Uso

1.4 Detalhamento de Caso de Uso

1.5 Perfis de Acesso

1.6 Regras de Negócio e Validações

1.1 Requisitos Funcionais

O que o sistema deve fazer: 16 requisitos (RF-01 a RF-16), um para cada caso de uso, com regras, critérios de aceitação, rotas da API e testes que os comprovam.

1.1.1 Convenções

- **Atores: Vendedor**, **Gerente** (herda tudo o que o Vendedor faz) e **Sistema** (regras automáticas).
- **Prioridade:** Essencial sem ela o PDV não opera · Importante necessária à gestão · Desejável melhora a operação.
- **Status:** todos os 16 requisitos estão **implementados e aprovados em teste** no backend (Ciclo 1.2, 320/320 testes).
- Valores monetários têm 2 casas decimais e arredondamento comercial (meio para cima). Datas seguem o fuso de Brasília (−03:00).

1.1.2 Lista de requisitos

ID	Requisito	Descrição	Ator	Prioridade
RF-01 UC01	**Autenticar no sistema**	O usuário entra com CPF e senha e recebe um token de acesso, que deve ser enviado em todas as demais requisições.	Vendedor, Gerente	Essencial
RF-02 UC02	**Abrir caixa**	No início do turno, o operador abre o próprio caixa informando o fundo de troco (suprimento inicial).	Vendedor, Gerente	Essencial
RF-03 UC03	**Fechar caixa**	No fim do turno, o operador informa o dinheiro contado; o sistema calcula o esperado, registra a diferença e fecha o caixa.	Vendedor, Gerente	Essencial
RF-04 UC04	**Registrar venda e consultar histórico**	O operador registra uma venda com um ou mais itens, identifica o cliente (ou usa o Consumidor Padrão) e a forma de pagamento.	Vendedor, Gerente	Essencial
RF-05 UC05	**Validar estoque automaticamente**	Durante o registro da venda, o sistema confere se há saldo para todos os itens antes de baixar qualquer um.	Sistema	Essencial
RF-06 UC06	**Aplicar cupom de desconto**	O operador informa o código de um cupom na venda; o desconto é calculado sobre o subtotal.	Vendedor, Gerente	Importante
RF-07 UC07	**Aplicar desconto direto do gerente**	Desconto em valor fixo (R$) concedido pelo Gerente, ou pelo Vendedor com a senha de um Gerente.	Gerente (ou Vendedor com senha do gerente)	Importante
RF-08 UC08	**Cadastrar e gerenciar clientes**	Cadastro de clientes Pessoa Física (CPF) ou Jurídica (CNPJ), usados na identificação da venda.	Vendedor, Gerente	Importante
RF-09 UC09	**Consultar estoque e alertas críticos**	Consulta de produtos com saldo atual, estoque mínimo e indicação de estoque crítico.	Vendedor, Gerente	Importante
RF-10 UC10	**Gerar relatório de comissões**	Consolida, por vendedor e período, quantidade de vendas, valor vendido e comissão acumulada.	Vendedor (própria), Gerente (todas)	Importante
RF-11 UC11	**Cancelar venda com estorno**	O Gerente cancela uma venda concluída; os itens voltam ao estoque e a venda sai dos totais.	Gerente	Essencial
RF-12 UC12	**Gerenciar catálogo de produtos**	O Gerente cadastra, altera, ajusta o estoque e inativa produtos.	Gerente	Essencial
RF-13 UC13	**Gerenciar funcionários e comissões**	O Gerente cadastra funcionários, define perfil e percentual de comissão e pode inativá-los.	Gerente	Essencial
RF-14 UC14	**Emitir e gerenciar cupons**	O Gerente cria cupons com período de validade; os vendedores consultam e usam.	Gerente	Desejável
RF-15 UC15	**Realizar sangria e suprimento de caixa**	O Gerente retira (sangria) ou adiciona (suprimento) dinheiro em espécie num caixa aberto.	Gerente	Essencial
RF-16 UC16	**Consultar logs de auditoria**	O Gerente consulta o histórico de ações sensíveis realizadas no sistema.	Gerente	Importante

Validações VAL-01 a VAL-08: seção 1.6. Regras, critérios de aceitação e testes de cada RF: `01-requisitos-funcionais.pdf`.

1.2 Requisitos Não Funcionais

Como o sistema deve se comportar: segurança, desempenho, consistência, auditoria, interoperabilidade, implantação e qualidade — com meta mensurável, forma de verificação e situação atual.

1.2.1 Resumo

ID	Requisito	Característica (ISO 25010)	Meta	Situação
RNF-01	Autenticação segura	Segurança	Senhas só em hash BCrypt; token assinado, válido por 24 h; bloqueio após tentativas erradas	**Atendido**
RNF-02	Autorização por perfil	Segurança	100% das operações administrativas negadas ao Vendedor	**Atendido**
RNF-03	Privacidade e proteção de dados (LGPD)	Segurança	Tráfego HTTPS; nenhum dado sensível em logs ou respostas de erro	**Parcial (certificado TLS no ambiente)**
RNF-04	Desempenho (VAL-07)	Eficiência de desempenho	p95 < 3 s em consulta de estoque e finalização de venda, inclusive com 20 operadores simultâneos	**Atendido (local)**
RNF-05	Consistência sob concorrência (VAL-08)	Confiabilidade	Nenhuma venda do mesmo saldo em duplicidade	**Atendido**
RNF-06	Integridade transacional e financeira	Confiabilidade	Operações atômicas; valores exatos em centavos	**Atendido**
RNF-07	Auditabilidade	Segurança (responsabilização)	100% das ações sensíveis registradas	**Atendido**
RNF-08	Interoperabilidade (API REST)	Compatibilidade	Contrato v2.3: JSON, RFC 7807, paginação padronizada	**Atendido**
RNF-09	Implantação em containers (Docker / AWS EC2)	Portabilidade	Subida com um comando (Docker Compose)	**Validado no Docker; AWS pendente**
RNF-10	Disponibilidade e tratamento de falhas	Confiabilidade	Health check público; falhas de banco → 503 sem detalhes; sessões e bloqueios preservados em reinícios	**Atendido**
RNF-11	Manutenibilidade e testabilidade	Manutenibilidade	Cobertura ≥ 80% das linhas e ≥ 70% dos ramos; esquema versionado	**Atendido**
RNF-12	Usabilidade	Usabilidade	Mensagens em português, claras e acionáveis; operação ágil no caixa	**Parcial (front-end)**

Os critérios, resultados medidos e testes de cada requisito estão no documento `02-requisitos-nao-funcionais.pdf`. Desde o Ciclo 1.2, o RNF-09 (Docker) foi executado e validado localmente pelo roteiro RT-DEP; falta a implantação na AWS.

1.3 Diagrama de Caso de Uso

1.3.1 Diagrama

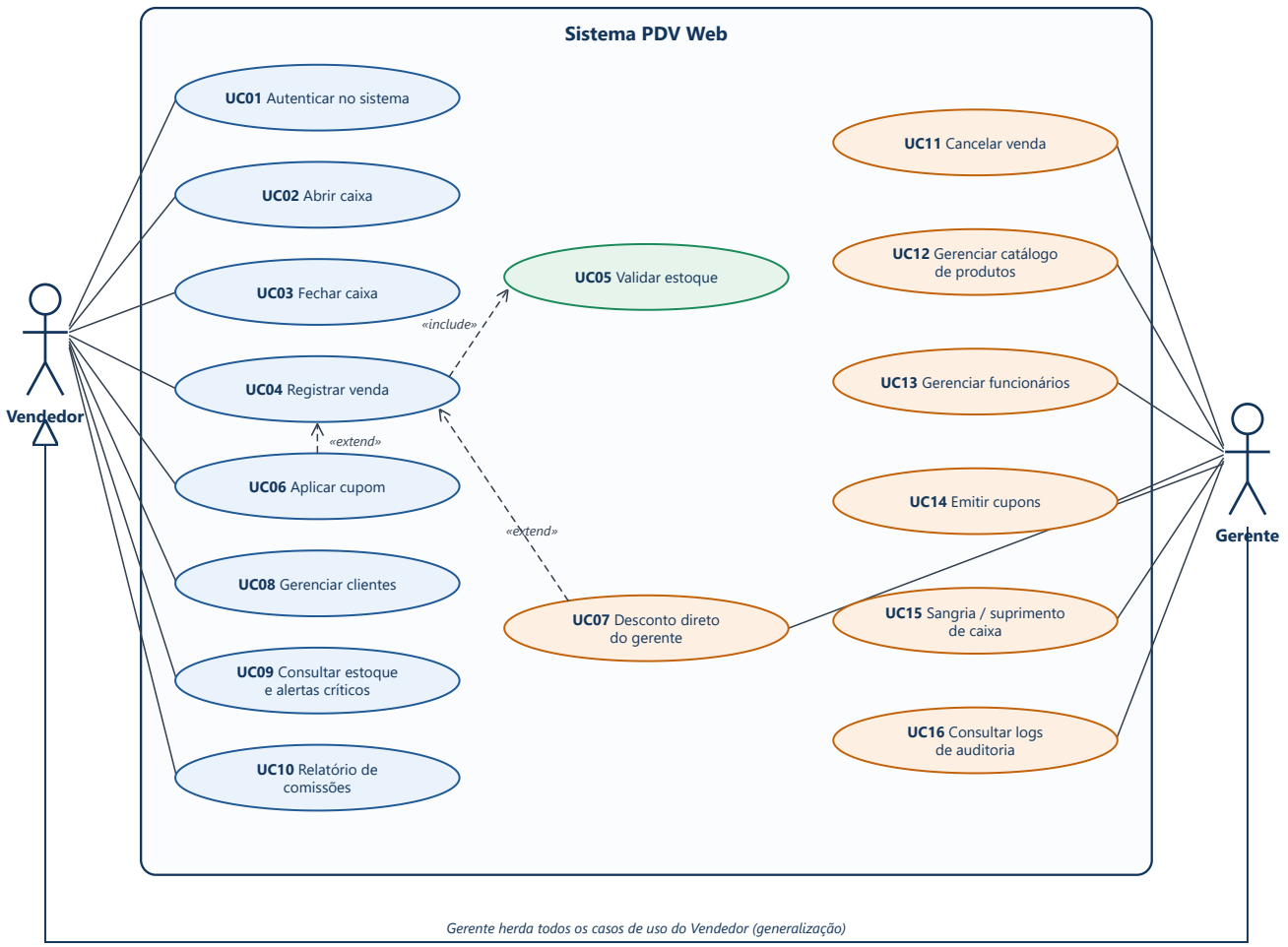


Figura 1 — Casos de uso do PDV Web. Azul: Vendedor e Gerente · laranja: exclusivos do Gerente · verde: executado automaticamente pelo Sistema.

1.3.2 Atores

Ator	Descrição
Vendedor	Operador da frente de caixa: abre e fecha o próprio caixa, registra vendas, aplica cupons, cadastra clientes, consulta estoque e a própria comissão.
Gerente	Herda todos os casos de uso do Vendedor e tem os privilégios administrativos: desconto direto, cancelamento com estorno, catálogo e estoque, funcionários e comissões, cupons, sangria/suprimento e auditoria.
Sistema	Executa regras automáticas durante as operações. No diagrama, aparece no UC05 (validação de estoque, em verde), incluído no registro da venda; por ser um caso de uso incluído, não tem ator desenhado.

1.4 Detalhamento de Caso de Uso

1.4.1 Dicionário de Casos de Uso

Inventário completo dos casos de uso do PDV Web, com o ator principal e os relacionamentos de cada um. A correspondência com os requisitos funcionais é direta: UCnn ↔ RF-nn.

UC	Nome	Ator principal	Relacionamentos e observações
UC01	Autenticar no sistema	Vendedor, Gerente	Pré-requisito de todos os demais
UC02	Abrir caixa	Vendedor, Gerente	Pré-requisito do UC04
UC03	Fechar caixa	Vendedor, Gerente	Vendedor fecha o próprio caixa; Gerente fecha qualquer um
UC04	Registrar venda	Vendedor, Gerente	**«include»** UC05; estendido por UC06 e UC07
UC05	Validar estoque	Sistema	Sempre executado dentro do UC04
UC06	Aplicar cupom de desconto	Vendedor, Gerente	**«extend»** UC04 — opcional, quando há código de cupom
UC07	Aplicar desconto direto do gerente	Gerente	**«extend»** UC04 — o Vendedor só pode solicitá-lo informando a senha de um Gerente
UC08	Cadastrar e gerenciar clientes	Vendedor, Gerente	Inativação exclusiva do Gerente
UC09	Consultar estoque e alertas críticos	Vendedor, Gerente	Somente leitura
UC10	Gerar relatório de comissões	Vendedor, Gerente	Vendedor: própria comissão · Gerente: todas
UC11	Cancelar venda	Gerente	Estorna os itens ao estoque
UC12	Gerenciar catálogo de produtos	Gerente	Inclui ajuste de estoque
UC13	Gerenciar funcionários	Gerente	Inclui parametrização do % de comissão
UC14	Emitir e gerenciar cupons	Gerente	Cupons usados no UC06
UC15	Realizar sangria / suprimento	Gerente	Requer caixa aberto (UC02)
UC16	Consultar logs de auditoria	Gerente	Registros gerados pelos demais casos de uso

Funcionalidades por caso de uso

Cada funcionalidade corresponde a um caso de uso (UC01–UC16) e a rotas da API v2.3.

Autenticação UC01

- Login por CPF (com ou sem máscara) e senha
- Token JWT válido por 24 h, com renovação (`refresh-token`)
- Logout invalida o token na hora, mesmo após reinício da API
- Troca de senha com conferência da senha atual (mín. 8 caracteres)
- Tentativas de login com falha registradas na auditoria; 5 erros bloqueiam por 15 min

`POST /auth/login · /refresh-token · /logout · PATCH /auth/alterar-senha`

Controle de caixa UC02 · UC03 · UC15

- Abertura com fundo de troco; só um caixa aberto por operador
- Saldo em dinheiro do turno em tempo real
- Sangria (retirada) e suprimento (reforço) com justificativa, só pelo gerente
- Sangria nunca maior que o dinheiro no caixa
- Fechamento: valor contado × esperado, com a diferença (sobra/falta)

`/caixas/abertura · /caixas/atual · /{id}/sangria · /{id}/suprimento · /{id}/fechamento`

Registro de venda UC04 · UC05

- Vários itens; o mesmo produto repetido é somado
- Cliente cadastrado ou "Consumidor Padrão"
- Pagamento: Dinheiro, PIX, Cartão de Crédito, Cartão de Débito ou Crediário
- Estoque validado e baixado na mesma transação; um item sem saldo cancela a venda inteira
- Preço sempre do catálogo (o enviado é ignorado)
- Resposta traz alertas de estoque crítico

`POST /vendas · GET /vendas · GET /vendas/{id}`

Descontos UC06 · UC07 · UC14

- Cupom percentual ou de valor fixo, com período de validade
- Desconto direto do gerente em R$ fixo
- Ordem: primeiro o cupom, depois o desconto direto sobre o restante
- Vendedor precisa informar a senha de um gerente; o gerente que autorizou fica registrado
- O total nunca fica negativo

`/cupons (CRUD) · campos cupom_codigo e desconto_direto_gerente na venda`

Cancelamento e estorno UC11

- Exclusivo do gerente, com motivo obrigatório
- Os itens voltam automaticamente ao estoque
- A venda sai dos totais do caixa e das comissões
- Venda cancelada não pode ser cancelada de novo
- Vendas são imutáveis: não existe edição nem exclusão

`POST /vendas/{id}/cancelar`

Catálogo e estoque UC09 · UC12

- SKU único, código de barras (EAN), unidade, fornecedor, NCM (8 dígitos) e CEST (7)
- Estoque atual e estoque mínimo; flag de estoque crítico
- Busca por nome ou SKU, filtro de itens críticos, paginação
- Ajuste de estoque e mudança de preço auditados (valor anterior e novo)
- Exclusão lógica: o histórico é preservado

`/produtos (CRUD, PUT e PATCH)`

Clientes UC08

- Pessoa Física (CPF) ou Jurídica (CNPJ), com validação dos dígitos
- Documento único; contato e endereço opcionais
- "Consumidor Padrão" protegido contra alteração
- Busca por documento ou nome

`/clientes (CRUD; inativação só pelo gerente)`

Funcionários e comissões UC10 · UC13

- Cadastro com CPF, perfil, senha e % de comissão (0 a 100)
- Comissão calculada em cada venda e congelada nela
- Relatório por período: total vendido e comissão acumulada
- Inativar ou mudar o perfil de alguém encerra as sessões dessa pessoa

`/usuarios (CRUD) · GET /vendas/comissoes`

Auditoria UC16 · Regra 1.2

Toda ação sensível gera um registro com **quem**, **quando**, **o quê** e os **valores anterior e novo**: login (sucesso e falha), abertura/fechamento de caixa, sangria, suprimento, venda, desconto do gerente, cancelamento, cadastros e alterações de produtos, preços, estoque, clientes, cupons e funcionários. Se a operação falhar, o registro também é desfeito: nunca há LOG de algo que não aconteceu. No banco, gatilhos tornam o LOG "somente inclusão".

`GET /auditoria?data_inicio=…&data_fim=…&tipo_acao=…`

1.4.2 Especificação e Descrição de Casos de Uso

Especificação detalhada dos casos de uso centrais da operação — registro de venda, desconto do gerente e cancelamento —, com o comportamento, as regras de negócio e os fluxos de interação entre os atores e o sistema.

1.4.2.1 Atores

Vendedor	Operador: abre e fecha o próprio caixa, cadastra clientes e realiza vendas.
Gerente	Privilégios administrativos: catálogo, cupons, descontos diretos, sangria/suprimento, cancelamentos, funcionários, auditoria e comissões consolidadas. Herda os casos de uso do Vendedor.
Sistema	Executa validações automáticas (estoque, descontos, permissões) e registra a auditoria.

Convenção das exceções: entre parênteses, o código HTTP e o tipo de erro devolvidos pela API.

1.4.2.2 Descrição detalhada dos casos de uso

UC04 — Registrar venda

RF-04

Ator principal	Vendedor, Gerente
Pré-condições	Caixa do próprio operador ABERTO; produtos ativos no catálogo.
Fluxo principal	1. O operador informa o caixa, o cliente (opcional) e a forma de pagamento (Dinheiro, PIX, Cartão de Crédito, Cartão de Débito ou Crediário). 2. Adiciona os itens (produto e quantidade); itens repetidos do mesmo produto são somados. 3. (Opcional) Aplica cupom (UC06) e/ou desconto direto (UC07). 4. O sistema valida o estoque (UC05). 5. O sistema calcula o subtotal com os preços do catálogo e aplica os descontos. 6. O sistema registra a venda e os itens e baixa o estoque. 7. O sistema calcula a comissão (valor líquido × percentual do vendedor) e congela o percentual na venda. 8. O sistema devolve o comprovante com status CONCLUIDA e os alertas de estoque crítico.
Fluxos alternativos	**A1 Sem cliente:** a venda é atribuída ao "CONSUMIDOR PADRÃO". **A2 Consultar histórico:** lista paginada por período, vendedor e status; detalhe com itens. O Vendedor vê só as próprias vendas.
Exceções	**E1** Caixa de outro operador (403); fechado (422 `caixa-fechado`); inexistente (404). **E2** Cliente ou produto inexistente/inativo (404). **E3** Sem itens, quantidade fora de 1–10.000 ou forma de pagamento inválida (400). **E4** Estoque insuficiente (422, ver UC05). **E5** Tentativa de editar ou excluir uma venda (405).
Pós-condições	Venda CONCLUIDA; estoque baixado; valor somado ao caixa e à comissão; venda auditada. Qualquer falha desfaz a operação inteira.

UC07 — Aplicar desconto direto do gerente

RF-07

Ator principal	Gerente; Vendedor somente com a senha de um Gerente (estende o UC04)
Regras de negócio	Valor fixo em R$, não percentual. Aplicado sobre o valor que sobra depois do cupom. O total da venda nunca fica negativo.
Fluxo principal	1. O operador informa o valor do desconto. 2. Se for Vendedor, informa também a senha de autorização de um Gerente ativo. 3. O sistema valida a permissão e deduz o valor do total. 4. O sistema grava na venda qual Gerente autorizou e registra o desconto na auditoria.
Exceções	**E1** Vendedor sem senha ou com senha inválida (403); senha errada é auditada (`AUTORIZACAO_FALHA`). **E2** Desconto maior que o valor restante (422 `desconto-invalido`). **E3** Valor negativo (400). **E4** 5 senhas de gerente erradas pelo mesmo vendedor em 15 min → autorização bloqueada por 15 min (429).
Exemplo	Subtotal R$ 250,00 − cupom 10% (R$ 25,00) − desconto direto R$ 10,00 = **R$ 215,00**; comissão de 5% = R$ 10,75.

UC11 — Cancelar venda

RF-11

Ator principal	Gerente
Pré-condições	Venda com status CONCLUIDA.
Fluxo principal	1. O Gerente seleciona a venda e informa o motivo. 2. O status muda para CANCELADA (vendas não são editadas nem excluídas). 3. O sistema devolve os itens ao estoque. 4. O sistema registra o cancelamento na auditoria (`CANCELAMENTO_VENDA`), com motivo e itens estornados.
Exceções	**E1** Vendedor tentando cancelar → "Acesso Negado" (403). **E2** Venda já cancelada (422 `venda-ja-cancelada`). **E3** Motivo ausente ou com menos de 5 caracteres (400). **E4** Venda inexistente (404).
Pós-condições	Estoque estornado uma única vez; a venda deixa de contar no caixa e nas comissões.

Os demais casos de uso (UC01 a UC03, UC05, UC06, UC08 a UC10 e UC12 a UC16) estão resumidos no Dicionário de Casos de Uso (1.4.1) e especificados por completo no documento `04-detalhamento-casos-de-uso.pdf` .

1.5 Perfis de Acesso

O sistema tem dois atores. O **Gerente** herda tudo o que o Vendedor faz e tem os privilégios administrativos. As permissões são verificadas **duas vezes**: no filtro de segurança (antes de ler a requisição) e nos controllers.

Operação	Vendedor	Gerente
Entrar no sistema, renovar sessão, trocar a própria senha	✓	✓
Abrir e fechar o próprio caixa	✓	✓ **(qualquer caixa)**
Registrar vendas e aplicar cupom	✓	✓
Conceder desconto direto em R$	**com senha do gerente**	✓
Cadastrar, consultar e editar clientes	✓	✓
Inativar clientes	✗	✓
Consultar estoque e cupons	✓	✓
Cadastrar, alterar e inativar produtos (ajuste de estoque)	✗	✓
Emitir e alterar cupons	✗	✓
Ver histórico de vendas e comissões	**só as próprias**	✓ **todas**
Cancelar venda (estorno ao estoque)	✗	✓
Sangria e suprimento de caixa	✗	✓
Gerenciar funcionários e percentual de comissão	✗	✓
Consultar LOG de auditoria	✗	✓

> **Primeiro acesso:** o primeiro gerente é criado automaticamente na subida da aplicação, a partir das variáveis `PDV_ADMIN_CPF` e `PDV_ADMIN_SENHA`. Nenhuma senha fica escrita no código. A partir dele, os demais funcionários são cadastrados pela API.

1.6 Regras de Negócio e Validações

As oito validações da Documentação do Projeto e como cada uma foi implementada:

ID	Validação	Como o sistema garante	Status
VAL-01	Bloqueio de saldo insuficiente	Os produtos da venda são bloqueados no banco (`SELECT … FOR UPDATE`) e conferidos antes de qualquer baixa. Resposta 422: "*Estoque atual é 1, mas foi solicitado 3.*" O banco ainda tem um `CHECK (estoque ≥ 0)` como última barreira.	**Completo**
VAL-02	Controle de perfil e acesso	Regras por rota no filtro de segurança, antes da leitura da requisição, e `@PreAuthorize` nos controllers. Vendedor só enxerga os próprios caixas, vendas e comissões.	**Completo**
VAL-03	Estorno no cancelamento	Cancelamento devolve as quantidades ao estoque, marca a venda como CANCELADA e a retira dos totais. Bloqueio de linha impede estorno em dobro.	**Completo**
VAL-04	Alerta de estoque crítico	Estoque ≤ mínimo → flag `alerta_estoque_critico` , filtro `estoque_critico=true` e lista de alertas na resposta da venda.	**Backend pronto**
VAL-05	Controle rigoroso de caixa	Venda exige caixa ABERTO do próprio operador. Um caixa aberto por operador (índice único no banco). Sangria limitada ao saldo em dinheiro.	**Completo**

ID	Validação	Como o sistema garante	Status
VAL-06	Segurança e criptografia (LGPD)	BCrypt, JWT assinado, bloqueio após tentativas erradas, logout persistente, dados de teste sintéticos, CPF mascarado nos logs. A API força HTTPS e envia HSTS; o proxy TLS (Caddy) foi validado localmente — falta o certificado no ambiente AWS.	**Certificado na AWS**
VAL-07	Desempenho (< 3 s)	Medido com 1.000 produtos e com 20 operadores simultâneos (149 req/s): p95 190 ms, pior requisição 1.565 ms. Falta medir na AWS.	**Atendido local**
VAL-08	Reserva de estoque	A reserva no carrinho ficou para a próxima fase. O objetivo da regra (nenhuma venda duplicada do mesmo saldo) é garantido na finalização da venda.	**Parcial**

Exemplos reais das Regras de Sistema (seção 3) — todos comprovados por teste

Operador	Situação	Resultado no sistema
Vendedor	Camiseta P: estoque 10, vende 2	**Venda concluída; estoque 8 no banco**
Vendedor	Tênis 41: estoque 1, pede 3	**Bloqueio por saldo insuficiente; nada é gravado**
Vendedor	Meia: estoque 3, mínimo 2, vende 1	**Venda concluída + alerta de estoque crítico**
Vendedor	Tenta cancelar a venda #102	**"Acesso Negado" (403)**
Gerente	Cancela a venda #102	**Venda cancelada; estoque estornado; LOG gravado**

Tabela de decisão dos descontos

Perfil	Cupom	Desconto direto	Senha do gerente	Resultado (subtotal R$ 250)
Vendedor	—	—	—	R$ 250,00
Vendedor	10%	—	—	R$ 225,00
Vendedor	—	R$ 10	ausente ou errada	**Recusado (403)**
Vendedor	10%	R$ 10	correta	**R$ 215,00**
Gerente	10%	R$ 10	dispensada	**R$ 215,00**
Qualquer	10%	> restante	—	**Recusado (422)**

2

Projeto

2.1 Diagrama Entidade-Relacionamento
2.2 Dicionário de Dados
2.3 Tecnologias
2.4 Contrato da API
2.5 Segurança

2.1 Diagrama Entidade-Relacionamento

O esquema é criado e versionado automaticamente pelo **Flyway** na subida da aplicação: `V1` cria as tabelas e restrições; `V2` cadastra o "Consumidor Padrão"; `V3` cria as tabelas de segurança (tokens revogados e tentativas de acesso); `V4` acrescenta gatilhos de imutabilidade, novos CHECKs e índices.

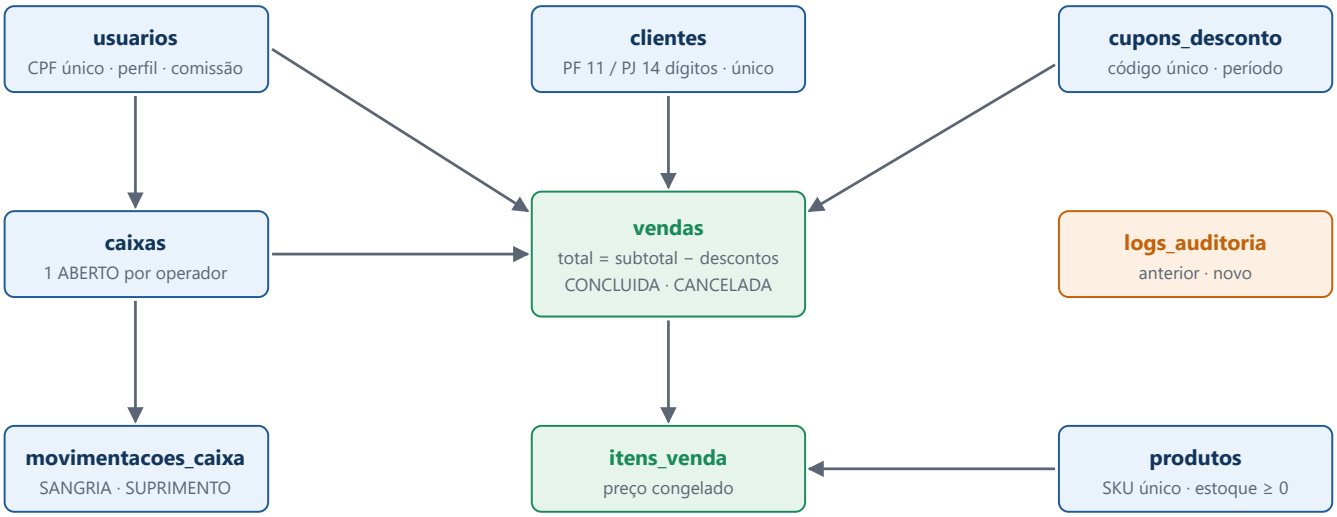


Figura 2 — Tabelas de negócio e relacionamentos principais. A seta sai da tabela referenciada e chega na tabela que guarda a chave estrangeira. As tabelas de segurança `tokens_revogados` e `tentativas_acesso` não têm chaves estrangeiras e aparecem no modelo lógico (2.1.1).

Relacionamentos

Tabela "um"	Tabela "muitos"	Chave estrangeira	Significado
usuarios	caixas	`usuario_id`	Operador dono do caixa (um ABERTO por vez)
usuarios	vendas	`usuario_id`	Vendedor que registrou a venda
usuarios	vendas	`gerente_autorizador_id` (opcional)	Gerente que autorizou o desconto direto
usuarios	vendas	`usuario_cancelamento_id` (opcional)	Gerente que cancelou a venda
usuarios	movimentacoes_caixa	`usuario_gerente_id`	Gerente que fez a sangria/suprimento
usuarios	logs_auditoria	`usuario_id` (opcional)	Autor da ação (nulo em tentativa de login com CPF inexistente)
caixas	movimentacoes_caixa	`caixa_id`	Sangrias e suprimentos do turno
caixas	vendas	`caixa_id`	Caixa em que a venda foi registrada
clientes	vendas	`cliente_id`	Cliente da venda (ou o Consumidor Padrão)
cupons_desconto	vendas	`cupom_id` (opcional)	Cupom aplicado
vendas	itens_venda	`venda_id` (exclusão em cascata)	Itens da venda
produtos	itens_venda	`produto_id`	Produto vendido

A chave `logs_auditoria.entidade_id` não é FK: guarda o id do registro afetado em qualquer tabela, identificada pela coluna `entidade`.

2.1.1 Modelo Lógico do Banco de Dados

Modelo físico do banco PostgreSQL 17 implantado pelas migrações Flyway V1 a V4: 11 tabelas, 102 colunas e 12 chaves estrangeiras, com os tipos de cada coluna.



Figura 3 — Entidades, colunas e relacionamentos. Notação "pé de galinha": traços duplos = lado "um"; três pontas = lado "muitos".

Regras garantidas pelo próprio banco

Restrição	Efeito
`CHECK (estoque_atual >= 0)`	Estoque nunca fica negativo, mesmo que a aplicação falhe
Índice único parcial em `caixas(usuario_id) WHERE status = 'ABERTO'`	Um único caixa aberto por operador

Restrição	Efeito
`CHECK (valor_total = valor_subtotal - desconto_cupom - desconto_gerente)`	Total da venda sempre coerente com os descontos
`CHECK` tipo × tamanho do documento	Cliente PF com 11 dígitos; PJ com 14
`CHECK` do cupom	Término posterior ao início; percentual até 100
`CHECK` de domínio	perfil, status, tipo e modalidade só aceitam os valores listados no dicionário de dados
`UNIQUE`	CPF do usuário, CPF/CNPJ do cliente, SKU e código de barras do produto, código do cupom
CHECKs da V4 (cancelamento, fechamento de caixa, subtotal do item, comissão)	Venda cancelada sempre com motivo, data e gerente; caixa fechado sempre com data; item com subtotal exato
Gatilhos da V4	Auditoria, itens e movimentações só aceitam inclusão; venda nunca é apagada nem tem valores alterados (só CONCLUIDA → CANCELADA); caixa fechado não muda
`CHECK (falhas >= 0)` + atualização atômica (`INSERT … ON CONFLICT … RETURNING`)	Tentativas simultâneas não escapam do limite de bloqueio

2.2 Dicionário de Dados

Dicionário de dados do banco do sistema, elaborado a partir do modelo lógico (seção 2.1.1). Legenda — **PK**: chave primária; **FK**: chave estrangeira; **UK**: valor único; **Obrig.**: campo NOT NULL. Valores em R$ usam `NUMERIC(10,2)`, datas `TIMESTAMPTZ`, e registros cadastrais são inativados (`ativo = false`), nunca apagados. Os papéis de acesso ao banco em produção estão no documento `06-dicionario-de-dados.pdf`.

Tabela: usuarios

funcionários do sistema

Campo	Tipo	Chave	Obrig.	Descrição
id	BIGSERIAL	PK	Sim	Identificador do usuário (Gerado pelo banco)
cpf	VARCHAR(11)	UK	Sim	CPF de login e auditoria (11 dígitos; CPF válido)
nome	VARCHAR(100)		Sim	Nome completo (Até 100 caracteres)
senha_hash	VARCHAR(100)		Sim	Senha criptografada; nunca exposta pela API (Hash BCrypt; a senha original tem de 8 a 72 caracteres)
perfil	VARCHAR(10)		Sim	Nível de acesso (VENDEDOR | GERENTE; padrão VENDEDOR)
percentual_comissao	NUMERIC(5,2)		Sim	% de comissão sobre as vendas (base líquida ou bruta, conforme PDV_COMISSAO_BASE) (0 a 100; padrão 0)
ativo	BOOLEAN		Sim	Falso = inativado; perde o acesso imediatamente (Padrão true)
criado_em	TIMESTAMPTZ		Sim	Data de cadastro (Padrão: agora)
atualizado_em	TIMESTAMPTZ		Não	Última alteração

Tabela: clientes

clientes PF e PJ

Campo	Tipo	Chave	Obrig.	Descrição
id	BIGSERIAL	PK	Sim	Identificador (Gerado pelo banco)
tipo	VARCHAR(2)		Sim	Pessoa física ou jurídica (PF | PJ)
cpf_cnpj	VARCHAR(14)	UK	Sim	Documento (PF: 11 dígitos (CPF válido) · PJ: 14 dígitos (CNPJ válido))
nome_razao_social	VARCHAR(150)		Sim	Nome ou razão social (Até 150 caracteres)
email	VARCHAR(150)		Não	Contato (Formato de e-mail)
telefone	VARCHAR(20)		Não	Contato (8 a 20 caracteres (dígitos, espaço, parênteses, + e −))
cep	VARCHAR(8)		Não	CEP (8 dígitos)
endereco	VARCHAR(200)		Não	Logradouro e número
bairro	VARCHAR(100)		Não	Bairro
cidade	VARCHAR(100)		Não	Cidade
uf	VARCHAR(2)		Não	Estado (2 letras maiúsculas)
ativo	BOOLEAN		Sim	Exclusão lógica (Padrão true)
criado_em	TIMESTAMPTZ		Sim	Data de cadastro (Padrão: agora)
atualizado_em	TIMESTAMPTZ		Não	Última alteração

Tabela: produtos

catálogo e estoque

Campo	Tipo	Chave	Obrig.	Descrição
id	BIGSERIAL	PK	Sim	Código interno (Gerado pelo banco)
sku	VARCHAR(40)	UK	Sim	Código do produto (Letras, números, ponto, hífen, sublinhado; gravado em maiúsculas)
nome	VARCHAR(120)		Sim	Descrição comercial (Até 120 caracteres)
fornecedor	VARCHAR(120)		Não	Fornecedor
unidade_medida	VARCHAR(10)		Sim	Unidade de venda (1 a 10 letras; padrão UN)
codigo_barras	VARCHAR(14)	UK	Não	EAN/GTIN (8 a 14 dígitos)
ncm	VARCHAR(8)		Não	Classificação fiscal (Exatamente 8 dígitos)
cest	VARCHAR(7)		Não	Código CEST (Exatamente 7 dígitos)
preco_venda	NUMERIC(10,2)		Sim	Preço praticado nas vendas (> 0)
estoque_atual	INTEGER		Sim	Quantidade disponível para venda (>= 0; padrão 0; até 1.000.000)
estoque_minimo	INTEGER		Sim	Ponto de alerta: crítico quando estoque_atual ≤ estoque_minimo (>= 0; padrão 0)
ativo	BOOLEAN		Sim	Exclusão lógica (produto fora de linha) (Padrão true)
criado_em	TIMESTAMPTZ		Sim	Data de cadastro (Padrão: agora)
atualizado_em	TIMESTAMPTZ		Não	Última alteração

Tabela: cupons_desconto

cupons promocionais

Campo	Tipo	Chave	Obrig.	Descrição
id	BIGSERIAL	PK	Sim	Identificador (Gerado pelo banco)
codigo	VARCHAR(30)	UK	Sim	Código digitado na venda (3 a 30 caracteres; maiúsculas)
tipo_desconto	VARCHAR(12)		Sim	Forma de cálculo (PERCENTUAL | VALOR_FIXO)
valor_desconto	NUMERIC(10,2)		Sim	Percentual ou valor em R$ (> 0; se PERCENTUAL, <= 100)
data_inicio_validade	TIMESTAMPTZ		Sim	Início da vigência (inclusive)
data_termino_validade	TIMESTAMPTZ		Sim	Fim da vigência (inclusive) (Posterior ao início)
ativo	BOOLEAN		Sim	Exclusão lógica (Padrão true)
criado_em	TIMESTAMPTZ		Sim	Data de emissão (Padrão: agora)
atualizado_em	TIMESTAMPTZ		Não	Última alteração

Tabela: caixas

sessões (turnos) de caixa

Campo	Tipo	Chave	Obrig.	Descrição
id	BIGSERIAL	PK	Sim	Identificador do caixa (Gerado pelo banco)
usuario_id	BIGINT	FK	Sim	Operador responsável (→ usuarios.id; um único caixa ABERTO por usuário (índice único parcial))
status	VARCHAR(7)		Sim	Situação do turno (ABERTO | FECHADO; padrão ABERTO)
suprimento_inicial	NUMERIC(10,2)		Sim	Fundo de troco (>= 0)
data_abertura	TIMESTAMPTZ		Sim	Abertura do turno (Padrão: agora)
data_fechamento	TIMESTAMPTZ		Não	Fim do turno (Preenchida no fechamento)
valor_final_informado	NUMERIC(10,2)		Não	Dinheiro contado pelo operador (Preenchido no fechamento)
saldo_esperado	NUMERIC(10,2)		Não	Suprimento + suprimentos + vendas em dinheiro − sangrias (Preenchido no fechamento)
diferenca	NUMERIC(10,2)		Não	Sobra (+) ou falta (−) (Informado − esperado)

Tabela: movimentacoes_caixa

sangrias e suprimentos

Campo	Tipo	Chave	Obrig.	Descrição
id	BIGSERIAL	PK	Sim	Identificador (Gerado pelo banco)
caixa_id	BIGINT	FK	Sim	Caixa afetado (→ caixas.id; o caixa precisa estar ABERTO)
usuario_gerente_id	BIGINT	FK	Sim	Gerente responsável (→ usuarios.id (perfil GERENTE))
tipo	VARCHAR(10)		Sim	Retirada ou entrada de dinheiro (SANGRIA | SUPRIMENTO)
valor	NUMERIC(10,2)		Sim	Valor movimentado (> 0; sangria ≤ saldo em dinheiro)
observacao	VARCHAR(255)		Sim	Justificativa (Não pode ser vazia)
data_hora	TIMESTAMPTZ		Sim	Momento da movimentação (Padrão: agora)

Tabela: vendas

cabeçalho das vendas

Campo	Tipo	Chave	Obrig.	Descrição
id	BIGSERIAL	PK	Sim	Número da venda (cupom) (Gerado pelo banco)
usuario_id	BIGINT	FK	Sim	Vendedor (base da comissão) (→ usuarios.id)
cliente_id	BIGINT	FK	Sim	Cliente ou Consumidor Padrão (→ clientes.id)
caixa_id	BIGINT	FK	Sim	Caixa da operação (→ caixas.id; caixa ABERTO do próprio vendedor)
cupom_id	BIGINT	FK	Não	Cupom aplicado, se houver (→ cupons_desconto.id)
gerente_autorizador_id	BIGINT	FK	Não	Gerente que autorizou o desconto direto (→ usuarios.id)
valor_subtotal	NUMERIC(10,2)		Sim	Soma dos itens a preço de catálogo (> 0)
desconto_cupom	NUMERIC(10,2)		Sim	Desconto do cupom (aplicado primeiro) (>= 0; padrão 0)
desconto_gerente	NUMERIC(10,2)		Sim	Desconto direto em R$ (>= 0; padrão 0)
valor_total	NUMERIC(10,2)		Sim	Valor pago (>= 0; = subtotal − descontos (CHECK))
modalidade_pagamento	VARCHAR(15)		Sim	Forma de pagamento (simulada) (DINHEIRO | PIX | CARTAO_CREDITO | CARTAO_DEBITO | CREDIARIO)
status	VARCHAR(10)		Sim	Situação da venda (CONCLUIDA | CANCELADA; padrão CONCLUIDA)
percentual_comissao	NUMERIC(5,2)		Sim	Percentual congelado (Copiado do vendedor no momento da venda)
valor_comissao	NUMERIC(10,2)		Sim	Comissão do vendedor (valor_total × percentual ÷ 100 (base LIQUIDO, padrão) ou valor_subtotal × percentual ÷ 100 (base BRUTO))
data_venda	TIMESTAMPTZ		Sim	Momento da venda (Padrão: agora)
motivo_cancelamento	VARCHAR(255)		Não	Preenchido no cancelamento (5 a 255 caracteres)
data_cancelamento	TIMESTAMPTZ		Não	Momento do cancelamento
usuario_cancelamento_id	BIGINT	FK	Não	Quem cancelou (→ usuarios.id (GERENTE))

Tabela: itens_venda

itens das vendas

Campo	Tipo	Chave	Obrig.	Descrição
id	BIGSERIAL	PK	Sim	Identificador do item (Gerado pelo banco)
venda_id	BIGINT	FK	Sim	Venda a que pertence (→ vendas.id (exclusão em cascata))
produto_id	BIGINT	FK	Sim	Produto vendido (→ produtos.id)
quantidade	INTEGER		Sim	Quantidade (> 0; até 10.000)
preco_unitario	NUMERIC(10,2)		Sim	Preço congelado (> 0; preço de catálogo no momento da venda)
subtotal	NUMERIC(10,2)		Sim	Valor do item (quantidade × preço unitário)

Tabela: logs_auditoria

registro das ações sensíveis

Campo	Tipo	Chave	Obrig.	Descrição
id	BIGSERIAL	PK	Sim	Identificador do registro (Gerado pelo banco)
usuario_id	BIGINT	FK	Não	Autor da ação (→ usuarios.id; nulo em login com CPF inexistente)
data_hora	TIMESTAMPTZ		Sim	Momento exato (Padrão: agora)
tipo_acao	VARCHAR(40)		Sim	Ação realizada (Um dos 25 tipos da seção 3)
entidade	VARCHAR(40)		Não	Tipo do registro afetado (USUARIO, PRODUTO, CLIENTE, CUPOM, CAIXA, VENDA)
entidade_id	BIGINT		Não	Id do registro afetado (Sem FK (aponta para a tabela indicada em entidade))
detalhes	VARCHAR(1000)		Sim	Descrição legível
valor_anterior	TEXT		Não	Estado antes da alteração
valor_novo	TEXT		Não	Estado depois da alteração

Tabela: tokens_revogados

sessões encerradas antes de expirar (V3)

Campo	Tipo	Chave	Obrig.	Descrição
jti	VARCHAR(64)	PK	Sim	Token revogado no logout ou na renovação (Identificador único do token (claim jti))
expira_em	TIMESTAMPTZ		Sim	Depois dela o registro é apagado pela limpeza periódica (de hora em hora) (Expiração original do token)
revogado_em	TIMESTAMPTZ		Sim	Momento do logout/renovação (Padrão: agora)

Tabela: tentativas_acesso

limite de tentativas contra força bruta (V3)

Campo	Tipo	Chave	Obrig.	Descrição
chave	VARCHAR(80)	PK	Sim	O que está sendo limitado (Prefixo cpf: (login por CPF), ip: (login por endereço) ou pin: (senha do gerente no desconto, por vendedor))
falhas	INTEGER		Sim	Tentativas erradas na janela atual (>= 0; padrão 0. Limites: 5 (CPF), 20 (IP), 5 (senha do gerente))
janela_inicio	TIMESTAMPTZ		Sim	Início da janela de contagem (A contagem recomeça após 15 min)
bloqueado_ate	TIMESTAMPTZ		Não	Até quando a chave recebe HTTP 429 (Preenchido ao atingir o limite (15 min))

Valores de `logs_auditoria.tipo_acao`

Grupo	Valores
Acesso	`LOGIN_SUCESSO`, `LOGIN_FALHA`, `SENHA_ALTERADA`, `ACESSO_BLOQUEADO` (limite de tentativas atingido)
Funcionários	`USUARIO_CREATE`, `USUARIO_UPDATE`, `USUARIO_INATIVACAO`
Produtos	`PRODUTO_CREATE`, `PRODUTO_UPDATE`, `PRODUTO_INATIVACAO`, `ESTOQUE_AJUSTE`
Clientes	`CLIENTE_CREATE`, `CLIENTE_UPDATE`, `CLIENTE_INATIVACAO`
Cupons	`CUPOM_CREATE`, `CUPOM_UPDATE`, `CUPOM_INATIVACAO`
Caixa	`CAIXA_ABERTURA`, `CAIXA_FECHAMENTO`, `SANGRIA`, `SUPRIMENTO`
Vendas	`VENDA_REGISTRO`, `DESCONTO_GERENTE`, `AUTORIZACAO_FALHA` (senha de gerente errada no desconto), `CANCELAMENTO_VENDA`

Índices

Índice	Coluna(s)	Finalidade
`ix_produtos_nome`	`lower(nome)`	Busca de produto por nome sem diferenciar maiúsculas
`ux_caixas_um_aberto_por_usuario`	`usuario_id` onde status = ABERTO	Único: um caixa aberto por operador
`ix_movimentacoes_caixa`	`caixa_id`	Totais de sangria/suprimento do turno
`ix_vendas_data`	`data_venda`	Histórico e relatórios por período
`ix_vendas_usuario`	`usuario_id`	Vendas e comissões por vendedor
`ix_vendas_caixa`	`caixa_id`	Totais do caixa
`ix_itens_venda_venda`	`venda_id`	Itens de uma venda
`ix_logs_auditoria_data` / `_tipo`	`data_hora` / `tipo_acao`	Filtros da consulta de auditoria
`ix_tokens_revogados_expira`	`expira_em`	Limpeza periódica dos tokens já expirados
`ix_caixas_usuario`, `ix_movimentacoes_gerente`, `ix_vendas_cliente`, `ix_vendas_cupom`, `ix_itens_venda_produto`, `ix_logs_auditoria_usuario`	Chaves estrangeiras (V4)	Junções e filtros por operador, cliente, cupom e produto sem varrer a tabela inteira
`ix_vendas_status_data`	`status`, `data_venda`	Relatórios de vendas concluídas e comissões por período
`ix_tentativas_janela`	`janela_inicio`	Limpeza das tentativas antigas

As colunas com `UNIQUE` (CPF, CPF/CNPJ, SKU, código de barras, código do cupom) também têm índice criado automaticamente pelo banco.

2.3 Tecnologias

Tecnologias utilizadas no PDV Web:

Java 25	Linguagem (OpenJDK, versão LTS)
Spring Boot 4.1	Back-end: Web MVC, Data JPA (Hibernate), Validation e Actuator
Spring Security + JWT	Segurança: tokens HS256 de 24 h, perfis e senhas BCrypt
PostgreSQL 17 + Flyway	Banco de dados e versionamento do esquema (migrações V1 a V4)
JUnit 6 · Mockito · Cucumber · JaCoCo · PIT	Testes unitários, de integração, BDD, cobertura e mutação
Docker · Docker Compose · Caddy	Infraestrutura: containers, backup diário e proxy HTTPS
AWS EC2	Hospedagem em nuvem (implantação pendente)
Maven Wrapper · GitHub Actions	Build e integração contínua
Front-end web	Telas do Vendedor e do Gerente — a definir (Ciclo 2)

As versões exatas de cada biblioteca e o motivo de cada escolha estão no documento `07-tecnologias.pdf`.

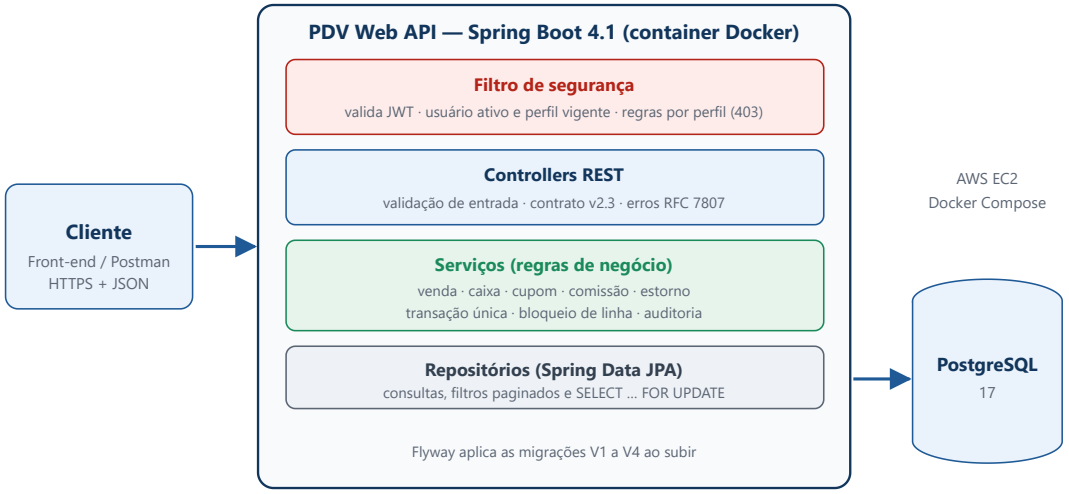


Figura 4 — Camadas da aplicação. Cada requisição passa pela segurança antes de qualquer regra de negócio.

Organização do código

Pacote	Responsabilidade
`auth`	login, refresh, logout, troca de senha
`usuario`	funcionários, perfis, comissões
`produto`	catálogo e estoque
`cliente`	clientes PF/PJ
`cupom`	cupons de desconto

Pacote	Responsabilidade
`caixa`	turnos, sangria, suprimento
`venda`	vendas, descontos, comissão, estorno
`auditoria`	LOG das ações sensíveis
`seguranca`	JWT, perfis, revogação
`comum`	erros, paginação, CPF/CNPJ, dinheiro

2.4 Contrato da API

A interface segue a **Documentação da API v2.3**: JSON em `snake_case`, listagens paginadas e erros no padrão RFC 7807. A documentação interativa (Swagger UI) fica em `/swagger-ui.html` no modo demonstração.

Padrão das listagens (paginação)

```
GET /produtos?pagina=1&limite=20&estoque_critico=true
{
  "pagina": 1, "limite": 20,
  "total_registros": 1, "total_paginas": 1,
  "dados": [{ "id": 104, "sku": "PROD-ELE-001",
     "estoque_atual": 2, "estoque_minimo": 3,
     "alerta_estoque_critico": true }]
}
```

Padrão de erros (RFC 7807)

```
// 422 Unprocessable Entity
{
  "type": ".../erros/estoque-insuficiente",
  "title": "Estoque Insuficiente",
  "status": 422,
  "detail": "Quantidade solicitada supera
            o estoque disponível.",
  "invalid_params": [{
    "name": "produto_id: 104",
    "reason": "Estoque atual é 2, mas foi
               solicitado 5."
  }]
}
```

Rotas por recurso

Recurso	Rotas	Perfil
Autenticação	`POST /auth/login` · `POST /auth/refresh-token` · `POST /auth/logout` · `PATCH /auth/alterar-senha`	público / autenticado
Produtos	`GET /produtos` · `GET /produtos/{id}` · `POST` · `PUT` · `PATCH` · `DELETE`	leitura: todos · escrita: Gerente
Clientes	`GET` · `POST` · `PUT` · `PATCH /clientes[/{id}]` · `DELETE /clientes/{id}`	todos · exclusão: Gerente
Cupons	`GET /cupons[/{id}]` · `POST` · `PUT` · `PATCH` · `DELETE`	leitura: todos · escrita: Gerente
Vendas	`POST /vendas` · `GET /vendas[/{id}]` · `GET /vendas/comissoes` · `POST /vendas/{id}/cancelar`	todos · cancelar: Gerente
Caixa	`POST /caixas/abertura` · `GET /caixas/atual` · `GET /caixas/{id}` · `POST /caixas/{id}/fechamento` · `/sangria` · `/suprimento`	todos · sangria/suprimento: Gerente
Usuários	`GET` · `POST` · `PUT` · `PATCH` · `DELETE /usuarios[/{id}]`	Gerente
Auditoria	`GET /auditoria`	Gerente
Saúde	`GET /actuator/health`	público

Códigos de resposta

Código	Quando acontece
200 / 201	Operação concluída / recurso criado
400	Dados inválidos, JSON malformado ou parâmetro fora do limite
401	Sem token, token inválido, expirado, revogado, ou usuário inativo/com perfil alterado
403	Perfil sem permissão, recurso de outro operador, desconto sem autorização
404 / 405	Recurso inexistente / método não permitido (ex.: editar uma venda)
409	Duplicidade (SKU, CPF/CNPJ, cupom) ou caixa já aberto

Código	Quando acontece
422	Regra de negócio: estoque insuficiente, caixa fechado, cupom inválido, desconto maior que o total, venda já cancelada
429	Tentativas de senha esgotadas (login ou senha do gerente): bloqueio de 15 min com `Retry-After`
503	Banco de dados indisponível (sem expor detalhes)

2.5 Segurança

Autenticação

- Senhas guardadas só como hash **BCrypt**
- Login com erro devolve sempre a mesma mensagem e leva o mesmo tempo: não revela se o CPF existe
- Token JWT assinado (HS256); a aplicação **não sobe** sem uma chave secreta forte
- Logout e renovação invalidam o token anterior

Autorização

- Perfis verificados em duas camadas
- Usuário inativado ou com perfil alterado perde o acesso **na hora**, sem esperar o token expirar
- Proteção contra acesso a dados de outro usuário trocando o ID na URL
- Desconto direto exige perfil de gerente ou a senha de um gerente ativo

Integridade financeira

- Preço vem sempre do catálogo; preço adulterado na requisição é ignorado
- Venda inteira numa única transação; qualquer falha desfaz tudo
- Restrições no banco: estoque ≥ 0, total = subtotal − descontos, documento único
- Vendas imutáveis; cancelamento só com motivo e auditoria; gatilhos impedem adulteração por SQL direto
- API conecta com um papel sem permissão de alterar tabelas (`pdv_app`)

Privacidade e erros

- Erros nunca expõem SQL, stack trace ou dados de conexão
- Banco indisponível → 503 genérico
- CPF mascarado nos registros de tentativas de login (LGPD)
- 5 senhas erradas bloqueiam o login por 15 min (429); o mesmo vale para a senha do gerente no desconto
- Logout gravado no banco: vale após reinício e entre servidores
- HTTPS obrigatório e HSTS ativáveis por configuração
- Só o health check é público; demais endpoints de monitoramento ficam fechados

As falhas de segurança encontradas pelos testes (DEF-WEB-001 a 005) e suas correções estão na seção 3.3.

3

Qualidade e Testes

3.1 Processo e Plano de Testes

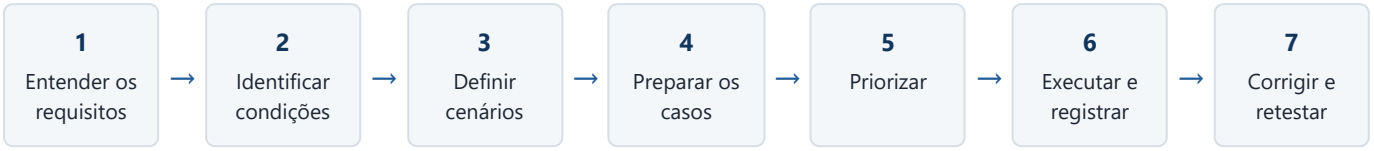
3.2 Resultados da Execução

3.3 Defeitos

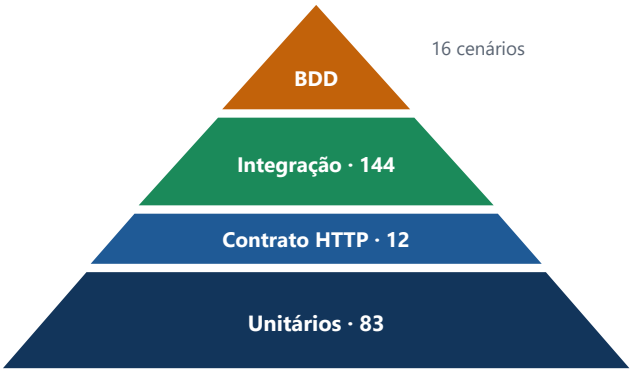
3.4 Cenários BDD e Rastreabilidade

3.1 Processo e Plano de Testes

Os testes seguiram as **7 etapas do Tutorial de Cenários de Testes** e o padrão IEEE 829 / ISO/IEC/IEEE 29119-3 de documentação.



Níveis de teste (pirâmide)



Casos distintos por nível (Ciclo 1.2)

Técnicas aplicadas

- **Partição de equivalência:** CPF/CNPJ, tipos de pessoa, formas de pagamento, perfis
- **Valores-limite:** preço 0,00 × 0,01; estoque = mínimo; desconto = total; sangria = saldo; vigência do cupom ao segundo
- **Tabela de decisão:** cupom × desconto direto × perfil × autorização
- **Transição de estados:** caixa ABERTO → FECHADO; venda CONCLUIDA → CANCELADA
- **Segurança:** token adulterado, outra chave, perfil sem permissão, troca de ID na URL, SQL injection, força bruta
- **Exceção:** banco indisponível, JSON malformado, erro inesperado
- **Concorrência:** 20 vendas simultâneas do último lote
- **Desempenho e carga:** 1.000 produtos; 20 operadores simultâneos
- **Mutação (PIT):** mede se os testes detectam erros inseridos no código

Critérios de saída do ciclo

#	Critério	Meta	Resultado	Situação
CS-1	Casos P1 executados e aprovados	100%	100%	✓
CS-2	Taxa de aprovação	≥ 95%	100% (320/320)	✓
CS-3	Defeitos Críticos/Altos abertos (backend)	0	0	✓
CS-4	Requisitos do escopo com ≥ 1 caso aprovado	100%	100% (38/38)	✓
CS-5	Cobertura de código	≥ 80% linhas, ≥ 70% ramos	96,6% / 87,6%	✓
CS-6	Tempo de resposta	p95 < 3.000 ms	p95 190 ms (local)	✓
CS-7	Força dos testes unitários (mutação)	≥ 80%	100%	✓

3.2 Resultados da Execução

Execução oficial do Ciclo 1.2: 28/10/2026, `./mvnw clean verify`, 3 min 34 s, Windows 10 Pro, OpenJDK 25, Spring Boot 4.1.2 e PostgreSQL 17 embarcado.

320	255	87,6%	38/39
execuções · 0 falhas	casos distintos (239 CT + 16 BDD)	dos ramos de decisão cobertos	requisitos com teste aprovado

Evolução por ciclo

Indicador	Ciclo 1 (26/10)	Ciclo 1.1 — robustez (28/10)	Ciclo 1.2 — banco (28/10)
Casos distintos	211	248	**255**
Execuções aprovadas	265/265	313/313	**320/320**
Cobertura (linhas / ramos)	94,1% / 81,2%	96,8% / 87,6%	**96,6% / 87,6%**
Força de mutação	100% (11 classes)	100% (12 classes, 216 mutantes)	100%
Carga (20 simultâneos)	—	p95 281 ms · máx. 981 ms · 148 req/s	**p95 190 ms · máx. 1.565 ms · 149 req/s**

Concorrência: 20 vendedores tentando vender ao mesmo tempo o último lote de 5 unidades → **exatamente 5 vendas**, 15 recusas e estoque final 0, repetido 10 vezes por execução. Dois cancelamentos simultâneos → estoque estornado uma única vez.

Teste de mutação: na primeira rodada, 14 erros inseridos no código passaram despercebidos (ex.: *apagar a baixa de estoque da venda*). Os testes foram reforçados até detectar **100%** deles.

Desempenho (limite VAL-07: 3.000 ms)

Operação	Média	p95	Máximo
Listar 1.000 produtos (página de 100)	41,0 ms	74 ms	77 ms
Buscar produto por nome	25,1 ms	36 ms	46 ms
Finalizar venda com 10 itens	45,5 ms	59 ms	62 ms
Consultar um produto	7,2 ms	9 ms	9 ms
Carga: 20 operadores simultâneos (400 requisições, 0 erros) — Ciclo 1.1	100,9 ms	281 ms	981 ms
Carga: 20 operadores simultâneos (149 req/s, 0 erros) — Ciclo 1.2	—	190 ms	1.565 ms

Execução fora dos testes automatizados

- **Servidor real:** a API subiu como servidor HTTP (modo demonstração) e passou num roteiro de 14 passos com `curl`: health check, Swagger, login dos dois perfis, venda do exemplo do contrato (R$ 215,00, comissão R$ 10,75), bloqueio por estoque (422), sangria negada ao vendedor (403), fechamento sem diferença, bloqueio de login (429), logout e cabeçalhos de segurança.
- **Docker (roteiro RT-DEP):** banco e API saudáveis com as migrações V1–V4; login antes e depois de reiniciar a API; logout persistente; proxy Caddy com HTTP → HTTPS, HSTS e TLS 1.3 (local); backup manual e automático verificados; restauração comprovada.

3.3 Defeitos

Origem	Encontrados	Corrigidos e retestados	Reteste pendente	Abertos (aceitos)
Backend `pdv-web-api` — produto	9	9	0	0
Backend — script de teste	1	1	0	0
Desktop `SistemaVendasEmpresa`	9	0	7	2
Total de defeitos de software	**19**	**10**	**7**	**2**

Por severidade: Crítica 1 · Alta 3 · Média 10 · Baixa 5. **Defeitos Críticos ou Altos abertos no backend: 0.** As 5 inconsistências da documentação (DOC-01 a DOC-05) foram resolvidas no ADR-001.

Defeitos do backend

ID	O que acontecia	Correção	Ciclo
DEF-WEB-001 Média	Vendedor que chamava rota de gerente com dados inválidos recebia 400 em vez de 403, descobrindo as regras de validação	Permissões checadas no filtro de segurança, antes de ler a requisição	1
DEF-WEB-002 Alta	Gerente rebaixado a vendedor mantinha os poderes por até 24 h (perfil gravado no token)	Token só vale se o perfil nele for igual ao perfil atual no banco	1
DEF-TST-001	Erro no próprio teste BDD: "10.00" lido como 1000 por causa do idioma português	Conversão de números do teste corrigida	1
DEF-WEB-003 Média	Logout era esquecido quando a API reiniciava: o token voltava a valer	Tokens revogados gravados no banco	1.1
DEF-WEB-004 Média	Login aceitava tentativas ilimitadas de senha (força bruta)	Bloqueio temporário por CPF e por IP, guardado no banco	1.1
DEF-WEB-005 Alta	Vendedor podia testar senhas de gerente sem limite pelo desconto da venda	5 erros bloqueiam por 15 min (429); cada erro é auditado	1.1
DEF-WEB-006	Primeira rajada após a subida passava de 3 s	Aquecimento do pool e das consultas antes de aceitar tráfego	1.1
DEF-WEB-007	Restauração de backup falhava no Git Bash e deixava a API parada	Script de restauração corrigido	1.2
DEF-WEB-008	`subir-docker.sh` pedia senha e terminava com erro com o ambiente saudável	Script corrigido	1.2
DEF-WEB-009	Com HTTPS obrigatório, o health check marcaria a API como "unhealthy"	Health check ajustado ao modo HTTPS	1.2

Todos corrigidos e retestados. Os defeitos do sistema desktop estão na Apresentação do Projeto (Do sistema desktop ao PDV Web). As lacunas de teste encontradas pelo teste de mutação (MUT-01 a MUT-13) foram todas fechadas.

3.4 Cenários BDD e Rastreabilidade

Os cenários de aceitação estão em **linguagem de negócio (Gherkin em português)** para que gerentes e vendedores possam validá-los sem ler código. Cada linha é executada pelo Cucumber contra a API completa e o PostgreSQL: 16 cenários distintos, 18 execuções e 130 passos aprovados.

Arquivo	Requisitos	Cenários
`venda_estoque.feature`	VAL-01, VAL-04, UC04, UC05	BDD-EST-01 (3 exemplos), BDD-EST-02

Arquivo	Requisitos	Cenários
`cancelamento_venda.feature`	VAL-02, VAL-03, UC11	BDD-CAN-01 a 03
`descontos.feature`	UC06, UC07	BDD-DES R1 a R8 (tabela de decisão)
`controle_caixa.feature`	VAL-05, UC02, UC03, UC15	BDD-CX-01 a 03

```
Esquema do Cenário: BDD-EST-01 Venda conforme o saldo em estoque - <produto>
  Dado que o produto "<codigo>" "<produto>" tem <estoque> unidades em estoque e mínimo de <minimo>
  Quando o vendedor registra a venda de <quantidade> unidades do produto "<codigo>"
  Então a API deve responder com o status <status>
  E o estoque do produto "<codigo>" deve ser de <estoque_final> unidades
  E o alerta de estoque crítico do produto "<codigo>" deve estar "<alerta>"
  | PROD-01 | Camiseta P | 10 | 2 | 2 | 201 | 8 | desligado |
  | PROD-02 | Tênis 41   |  1 | 0 | 3 | 422 | 1 | desligado |
  | PROD-03 | Meia       |  3 | 2 | 1 | 201 | 2 | ligado    |
```

Resumo da matriz de rastreabilidade

Grupo	Requisitos	Coberto	Parcial	Não coberto
Casos de uso	16	16	0	0
Validações VAL	8	4	4	0
Regras de sistema	5	4	1	0
Contrato da API	7	7	0	0
Não funcionais	3	2	0	1
Total	**39**	**33 (84,6%)**	**5 (12,8%)**	**1 (2,6%)**

Requisitos com pelo menos um caso executado e aprovado: **38 de 39 (97,4%)**. O único sem teste automatizado é o RNF-01 (Docker/AWS), validado no Docker pelo roteiro manual RT-DEP; a implantação na AWS segue pendente.